



University of Asia Pacific

Course title: Software Engineering Lab

Course Code : CSE 314

Sohoj Biniyog

Team Members

Shahriar Alam Shimul	22201227
Salman Nur Ramim	22201239
Tahsin Sheikh	22201243

Submitted To :

Noor Mairukh Khan Arnob

Lecturer

Department of Computer Science & Engineering

University of Asia Pacific

1. Introduction

The Sohoj Biniyog project is designed to provide a Shariah-compliant investment and funding platform that connects small and medium-sized enterprises (SMEs) with potential investors. This system aims to create an easy, transparent, and secure environment where entrepreneurs can raise funds for their businesses while investors can explore safe and ethical investment opportunities. By integrating a modern backend with a responsive frontend, Sohoj Biniyog ensures seamless data flow, user-friendly navigation, and trustworthy financial interactions. This project demonstrates how technology can bridge the gap between SMEs seeking capital and individuals looking for sustainable, impactful investment options.

2. Objectives

The primary objectives of this project are:

- To develop a backend system using Django Framework for managing investment campaigns, user accounts, and transaction data.
- To build a frontend using HTML, CSS.
- To provide a simple and interactive interface for investors and SMEs to connect and manage funding activities.
- To establish a secure and Shariah-compliant platform that ensures transparency, trust, and accessibility in SME financing.

3. Project Overview

The project is divided into two main components:

Backend: Developed using Django Framework, the backend provides managing users, campaigns, investments, and related financial transactions. It also enforces business logic such as campaign funding limits, validation.

Frontend: Built using HTML, CSS. It allows users to browse campaigns, track investments, and manage accounts through an interactive interface.

The system architecture ensures a clear separation of concerns, where the backend handles data and logic, and the frontend focuses on presentation and user interaction.

4. Technologies Used

The project utilizes modern web development technologies, including:

- **Backend:** Django Framework
- **Frontend:** HTML

- **Styling:** Tailwind CSS for responsive and modern UI design
- **Database:** MySQL (for production) and SQLite (for development)

5. System Design

The system is designed to be modular, scalable, and secure. Key design considerations include:

- **Database Design:** Models are structured to store information about users, SMEs, campaigns, and investments efficiently. Relationships are established using foreign keys to maintain integrity and traceability of transactions.
- **Frontend Design:** Components are modular, reusable, and responsive. Data fetched from the backend is displayed dynamically, ensuring a seamless experience for investors and SMEs.
- **Security:** Role-based access control ensures that SMEs, investors, and admins have distinct permissions, protecting sensitive financial data.

6. Implementations

In the current stage of the project, the following has been implemented:

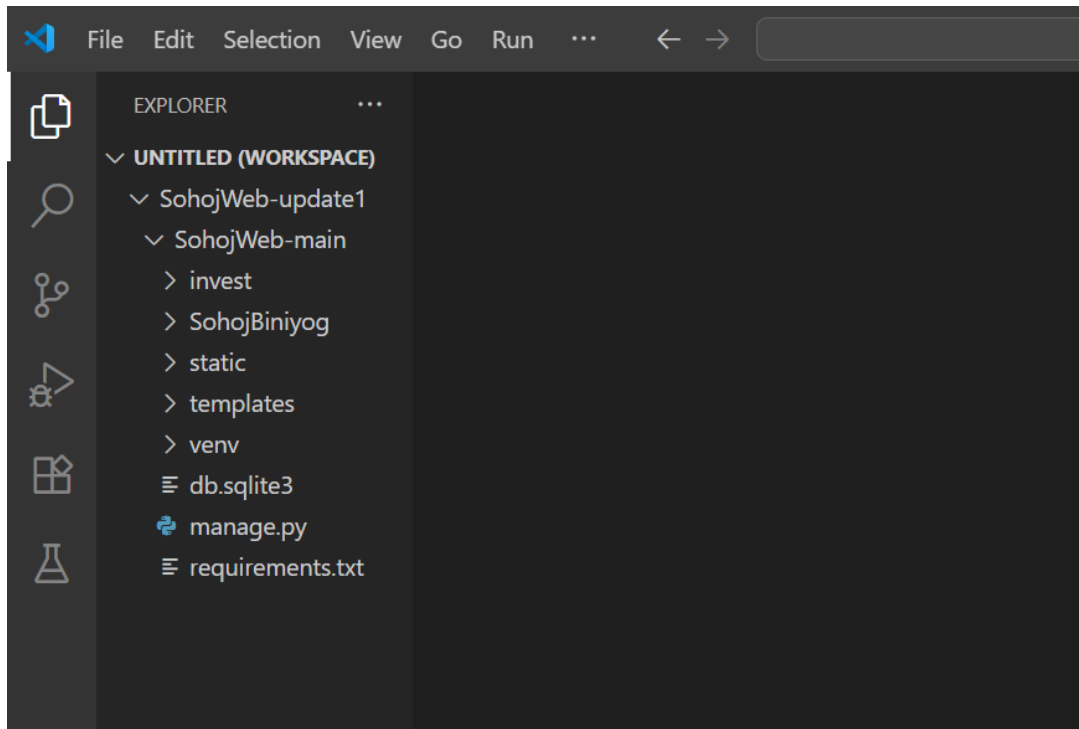
- **Backend Setup:** A Django project with models for users and campaigns has been created. Initial API endpoints return test data for integration checks.
- **Frontend Setup:** A Next.js application is configured to fetch data from the backend using Axios. The homepage displays campaigns and messages from the API to confirm successful integration.
- **Navigation:** A responsive navigation bar and footer have been implemented, allowing smooth transitions between pages such as Home, Funded Campaigns, About Us, and Contact.

Future implementation will focus on:

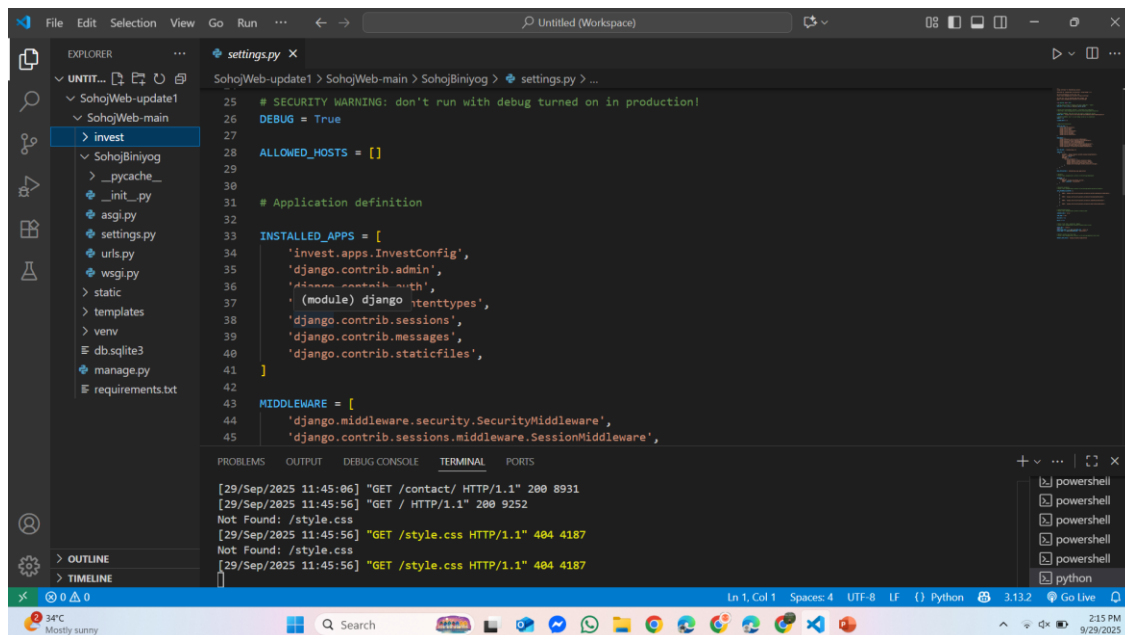
- Full CRUD operations for campaigns and investments
- Secure user authentication with role-based access (Investor, SME, Admin)
- Payment gateway integration (e.g., bKash/Nagad) for transactions
- Enhanced UI for campaign browsing, funding, and investor dashboards
- Detailed analytics and reporting for users and SMEs

7. Screenshot

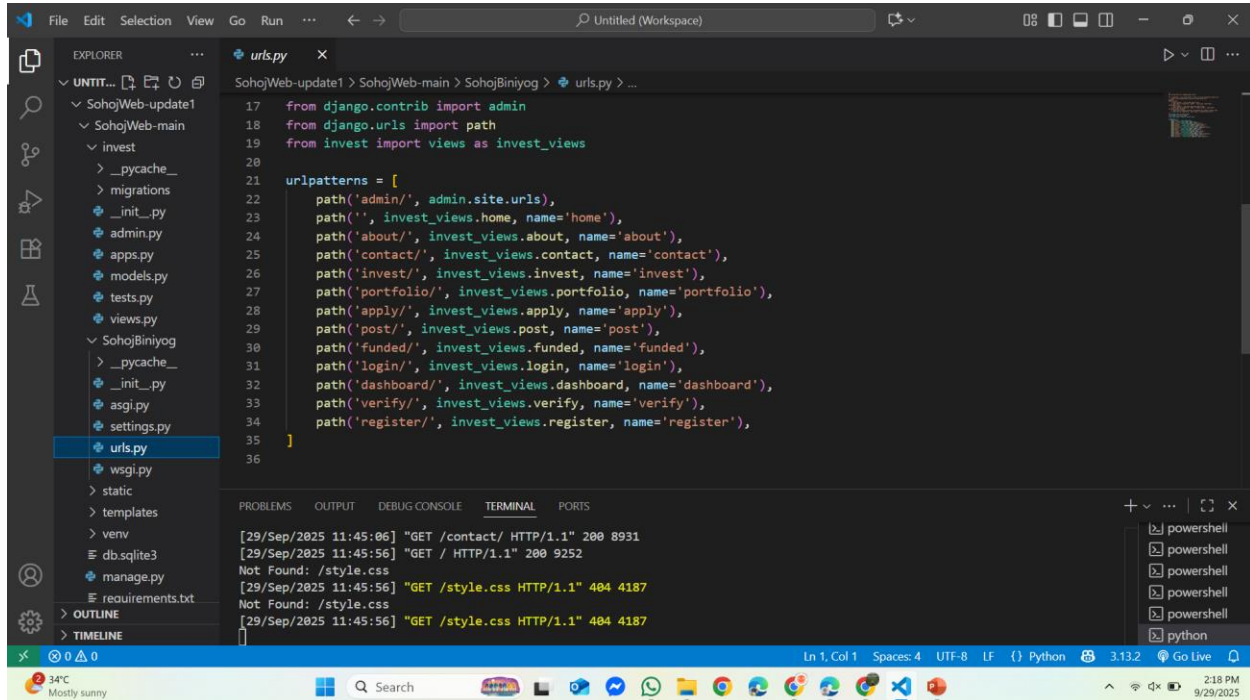
Main Directory



Django Settings



Django Urls



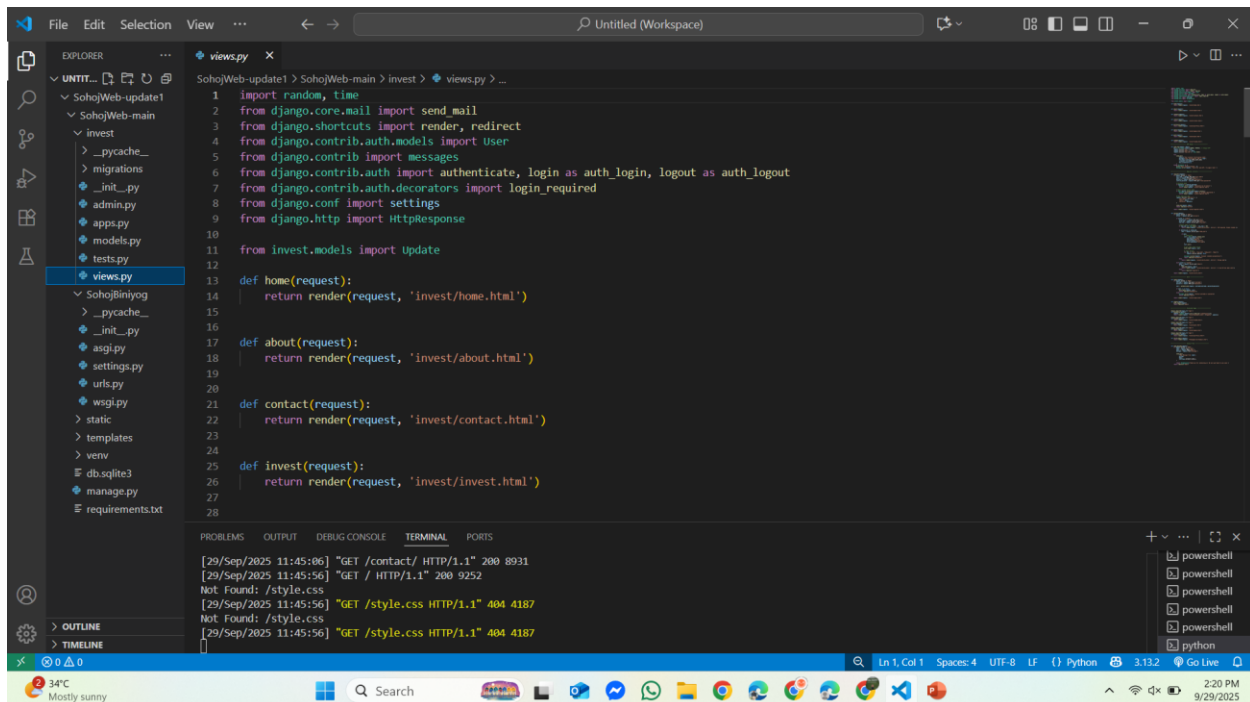
The screenshot shows the Visual Studio Code editor with a Django project. The Explorer panel on the left shows the project structure, with `urls.py` selected under the `invest` app. The main editor displays the content of `urls.py`, which imports `admin`, `path`, and `invest_views` from `django.contrib`, `django.urls`, and `invest` respectively. It then defines a list of `urlpatterns` using `path()` to map URLs to views. The terminal at the bottom shows the output of a Django server, including GET requests for `/contact/` and `/style.css`.

```
17 from django.contrib import admin
18 from django.urls import path
19 from invest import views as invest_views
20
21 urlpatterns = [
22     path('admin/', admin.site.urls),
23     path('', invest_views.home, name='home'),
24     path('about/', invest_views.about, name='about'),
25     path('contact/', invest_views.contact, name='contact'),
26     path('invest/', invest_views.invest, name='invest'),
27     path('portfolio/', invest_views.portfolio, name='portfolio'),
28     path('apply/', invest_views.apply, name='apply'),
29     path('post/', invest_views.post, name='post'),
30     path('funded/', invest_views.funded, name='funded'),
31     path('login/', invest_views.login, name='login'),
32     path('dashboard/', invest_views.dashboard, name='dashboard'),
33     path('verify/', invest_views.verify, name='verify'),
34     path('register/', invest_views.register, name='register'),
35 ]
36
```

Terminal Output:

```
[29/Sep/2025 11:45:06] "GET /contact/ HTTP/1.1" 200 8931
[29/Sep/2025 11:45:56] "GET / HTTP/1.1" 200 9252
Not Found: /style.css
[29/Sep/2025 11:45:56] "GET /style.css HTTP/1.1" 404 4187
Not Found: /style.css
[29/Sep/2025 11:45:56] "GET /style.css HTTP/1.1" 404 4187
```

Django Views



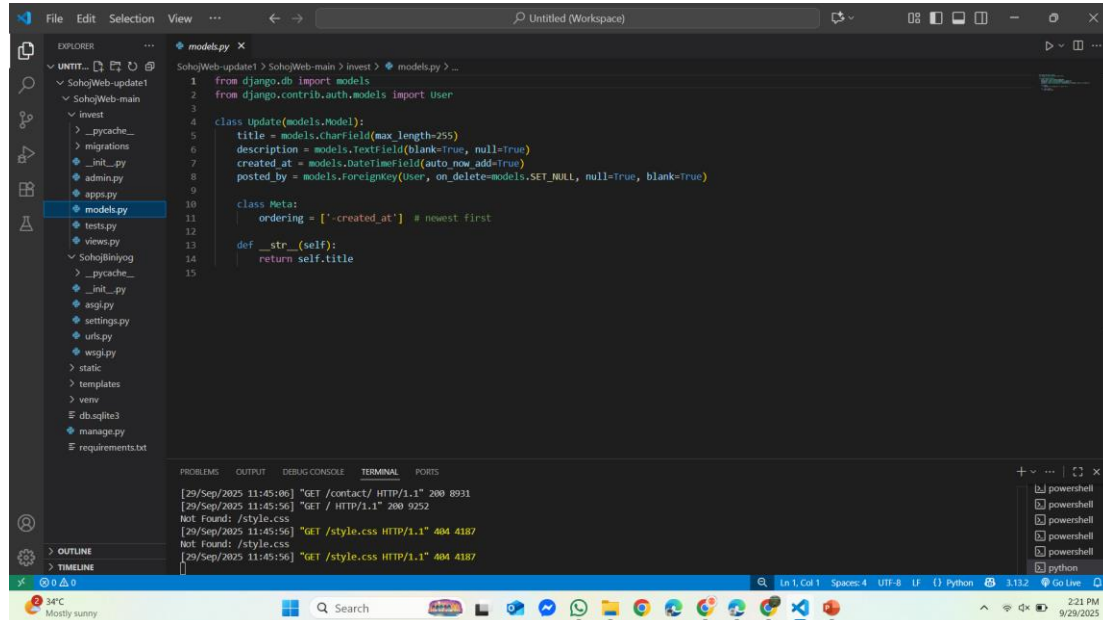
The screenshot shows the Visual Studio Code editor with a Django project. The Explorer panel on the left shows the project structure, with `views.py` selected under the `invest` app. The main editor displays the content of `views.py`, which imports various Django modules and defines four view functions: `home`, `about`, `contact`, and `invest`. Each function takes a `request` object and returns a `render` call to serve a specific HTML template. The terminal at the bottom shows the output of a Django server, including GET requests for `/contact/` and `/style.css`.

```
1 import random, time
2 from django.core.mail import send_mail
3 from django.shortcuts import render, redirect
4 from django.contrib.auth.models import User
5 from django.contrib import messages
6 from django.contrib.auth import authenticate, login as auth_login, logout as auth_logout
7 from django.contrib.auth.decorators import login_required
8 from django.conf import settings
9 from django.http import HttpResponse
10
11 from invest.models import Update
12
13 def home(request):
14     return render(request, 'invest/home.html')
15
16
17 def about(request):
18     return render(request, 'invest/about.html')
19
20
21 def contact(request):
22     return render(request, 'invest/contact.html')
23
24
25 def invest(request):
26     return render(request, 'invest/invest.html')
27
28
```

Terminal Output:

```
[29/Sep/2025 11:45:06] "GET /contact/ HTTP/1.1" 200 8931
[29/Sep/2025 11:45:56] "GET / HTTP/1.1" 200 9252
Not Found: /style.css
[29/Sep/2025 11:45:56] "GET /style.css HTTP/1.1" 404 4187
Not Found: /style.css
[29/Sep/2025 11:45:56] "GET /style.css HTTP/1.1" 404 4187
```

Django Models



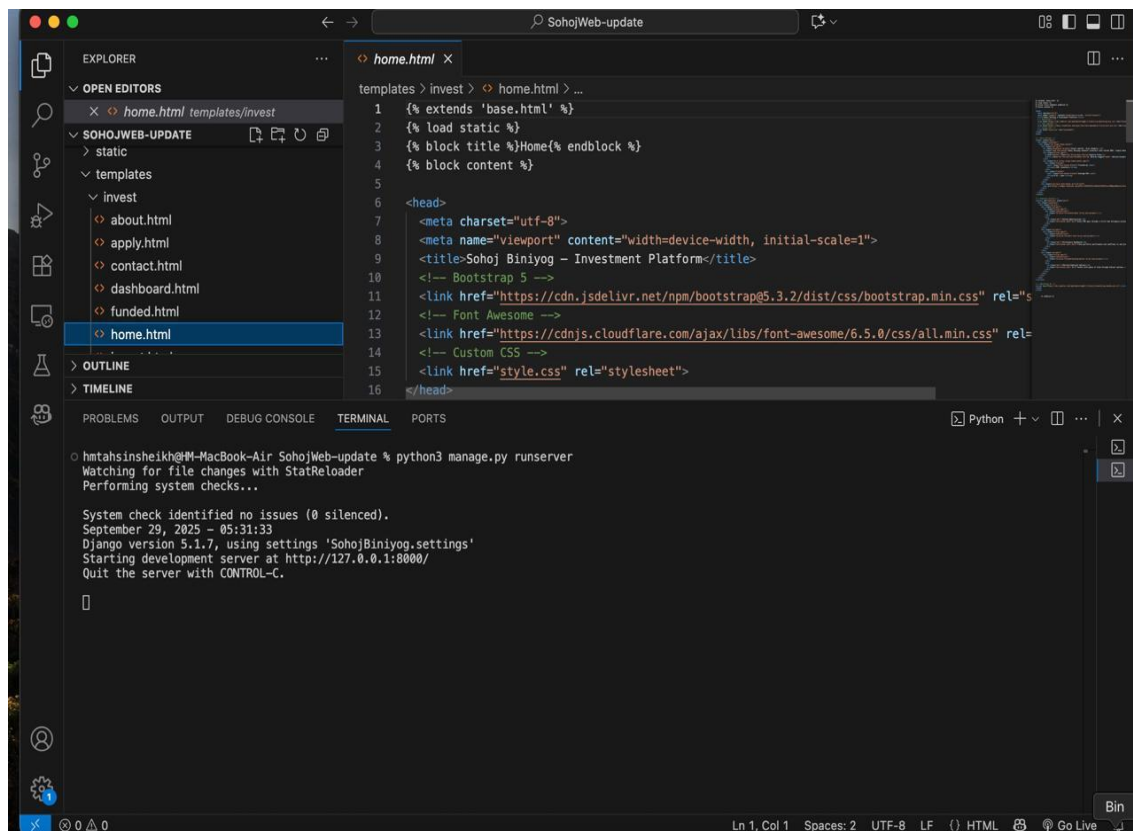
The screenshot shows a Visual Studio Code editor with a Django project. The Explorer sidebar on the left shows the project structure, with 'models.py' selected under the 'invest' directory. The main editor window displays the content of 'models.py', which defines a Django model named 'Update'. The model has the following fields: 'title' (CharField, max_length=255), 'description' (TextField, blank=True, null=True), 'created_at' (DateTimeField, auto_now_add=True), and 'posted_by' (ForeignKey to User, on_delete=models.SET_NULL, null=True, blank=True). The model's Meta class specifies 'ordering = ['-created_at']' for newest first. The __str__ method returns 'self.title'. The bottom panel shows the TERMINAL with network logs for GET requests to /contact/ and /style.css.

```
1 from django.db import models
2 from django.contrib.auth.models import User
3
4 class Update(models.Model):
5     title = models.CharField(max_length=255)
6     description = models.TextField(blank=True, null=True)
7     created_at = models.DateTimeField(auto_now_add=True)
8     posted_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True)
9
10    class Meta:
11        ordering = ['-created_at'] # newest first
12
13    def __str__(self):
14        return self.title
15
```

Terminal Output:

```
[29/Sep/2025 11:45:08] "GET /contact/ HTTP/1.1" 200 8931
[29/Sep/2025 11:45:56] "GET / HTTP/1.1" 200 9252
Not Found: /style.css
[29/Sep/2025 11:45:56] "GET /style.css HTTP/1.1" 404 4187
Not Found: /style.css
[29/Sep/2025 11:45:56] "GET /style.css HTTP/1.1" 404 4187
```

Home page



The screenshot shows a Visual Studio Code editor with a Django project. The Explorer sidebar on the left shows the project structure, with 'home.html' selected under the 'invest' directory. The main editor window displays the content of 'home.html', which is a Django template extending 'base.html'. It includes a title 'Sohoj Biniyog - Investment Platform' and links to Bootstrap 5, Font Awesome, and a custom CSS file. The bottom panel shows the TERMINAL with the output of the Django development server.

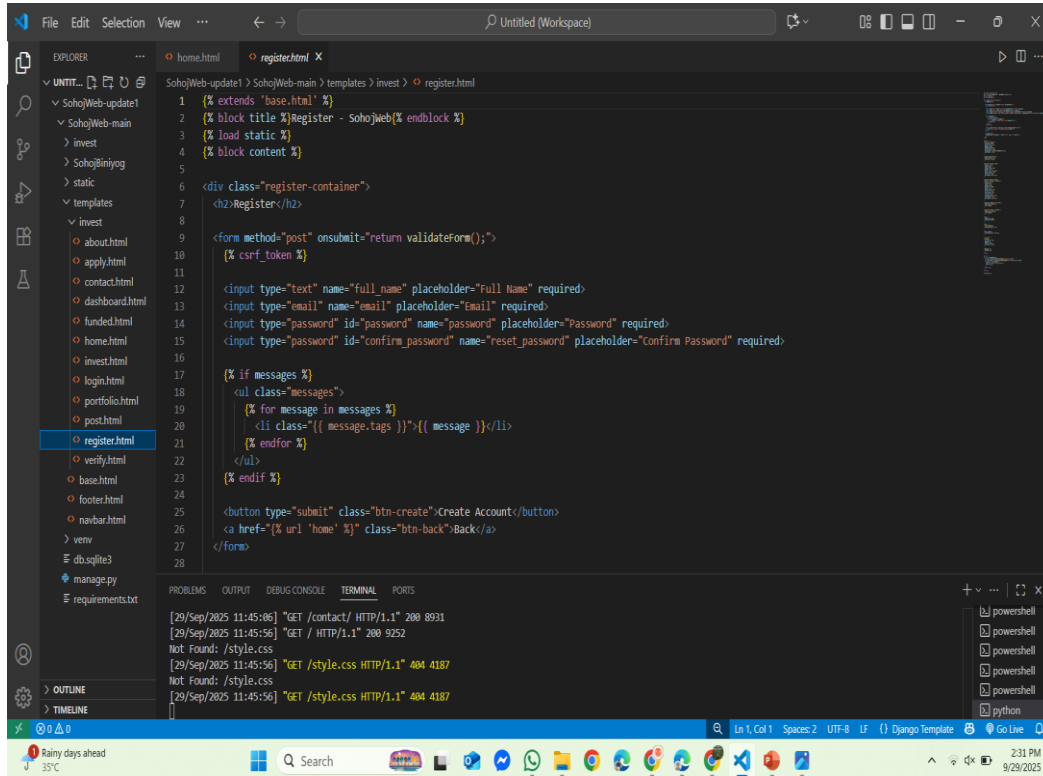
```
1 {% extends 'base.html' %}
2 {% load static %}
3 {% block title %}Home{% endblock %}
4 {% block content %}
5
6 <head>
7     <meta charset="utf-8">
8     <meta name="viewport" content="width=device-width, initial-scale=1">
9     <title>Sohoj Biniyog - Investment Platform</title>
10    <!-- Bootstrap 5 -->
11    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
12    <!-- Font Awesome -->
13    <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css" rel="stylesheet">
14    <!-- Custom CSS -->
15    <link href="style.css" rel="stylesheet">
16 </head>
```

Terminal Output:

```
hmtahinsheikh@M-MacBook-Air SohojWeb-update % python3 manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
September 29, 2025 - 05:31:33
Django version 5.1.7, using settings 'SohojBiniyog.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

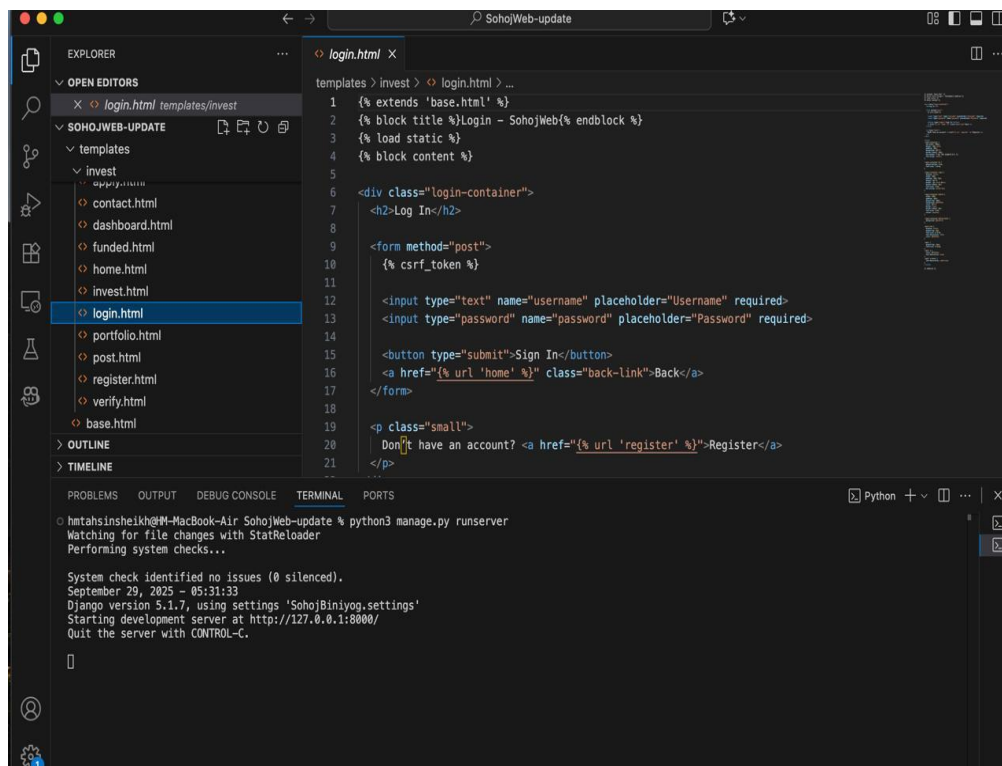
Register page



The screenshot shows the Visual Studio Code editor with the 'register.html' file open. The file is located in the 'templates > invest' directory. The code is a Django template that extends 'base.html'. It includes a title 'Register - SohojWeb' and a form for user registration. The form has fields for full name, email, password, and confirm password, all required. It also includes a CSRF token field. A message block is present for displaying messages. The form has a 'Create Account' button and a 'Back' link to the home page.

```
1 {% extends 'base.html' %}
2 {% block title %}Register - SohojWeb{% endblock %}
3 {% load static %}
4 {% block content %}
5
6 <div class="register-container">
7   <h2>Register</h2>
8
9   <form method="post" onsubmit="return validateForm();">
10     {% csrf_token %}
11
12     <input type="text" name="full_name" placeholder="Full Name" required>
13     <input type="email" name="email" placeholder="Email" required>
14     <input type="password" id="password" name="password" placeholder="Password" required>
15     <input type="password" id="confirm_password" name="reset_password" placeholder="Confirm Password" required>
16
17     {% if messages %}
18     <ul class="messages">
19       {% for message in messages %}
20         <li class="{{ message.tags }}">{{ message }}</li>
21       {% endfor %}
22     </ul>
23     {% endif %}
24
25     <button type="submit" class="btn-create">Create Account</button>
26     <a href="{% url 'home' %}" class="btn-back">Back</a>
27   </form>
28
```

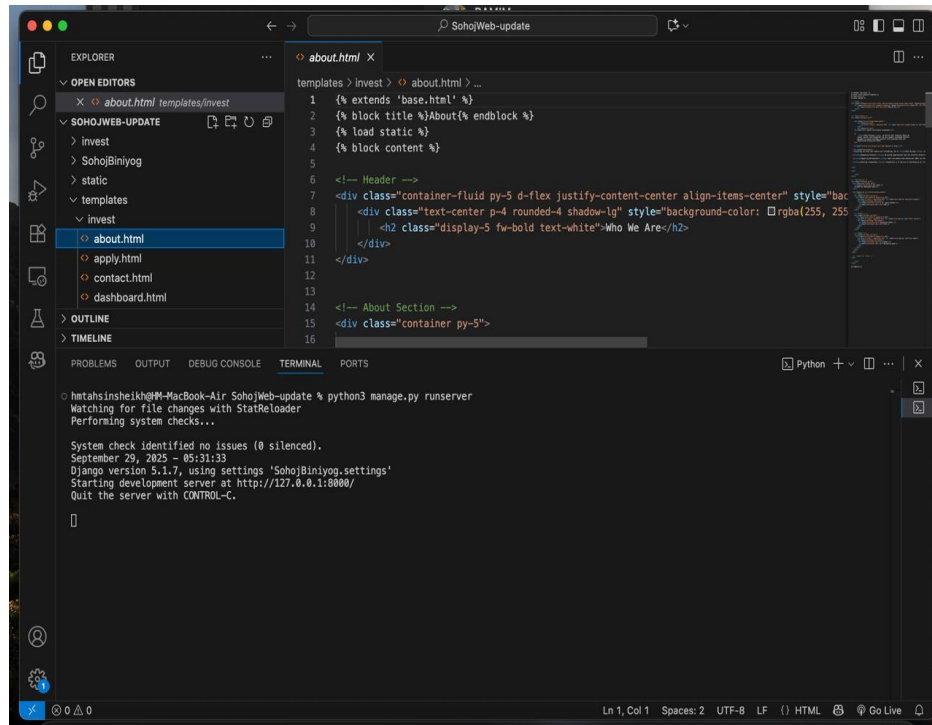
Log in page



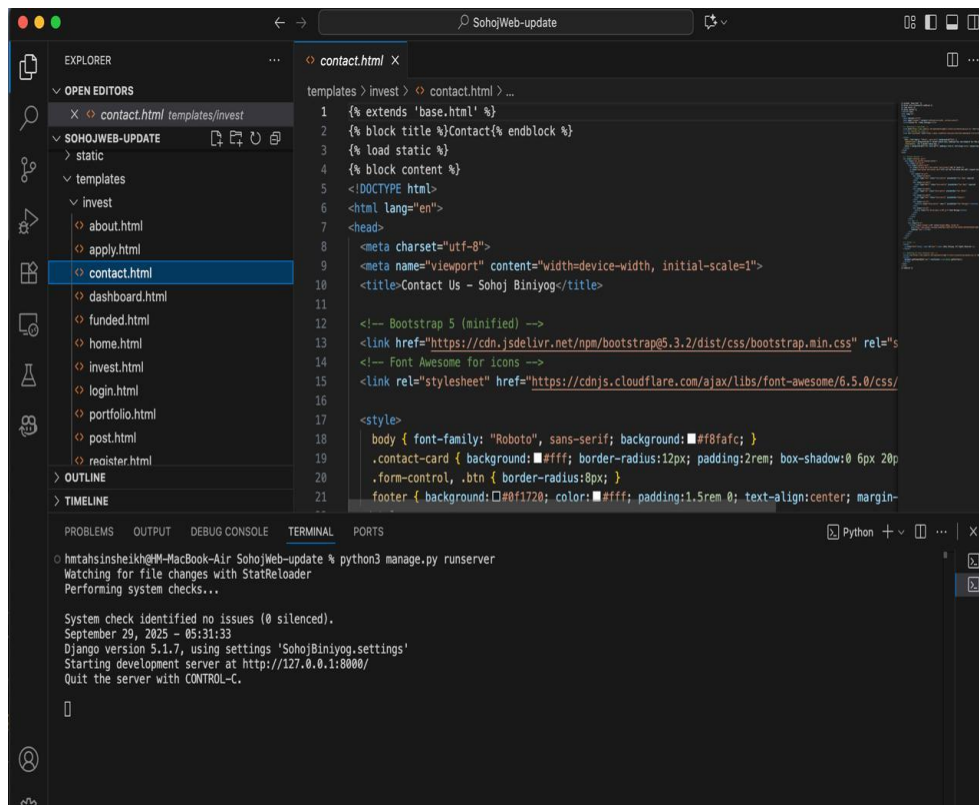
The screenshot shows the Visual Studio Code editor with the 'login.html' file open. The file is located in the 'templates > invest' directory. The code is a Django template that extends 'base.html'. It includes a title 'Login - SohojWeb' and a form for user login. The form has fields for username and password, both required. It also includes a CSRF token field. The form has a 'Sign In' button and a 'Back' link to the home page. Below the form, there is a link to the 'Register' page for users who do not have an account.

```
1 {% extends 'base.html' %}
2 {% block title %}Login - SohojWeb{% endblock %}
3 {% load static %}
4 {% block content %}
5
6 <div class="login-container">
7   <h2>Log In</h2>
8
9   <form method="post">
10     {% csrf_token %}
11
12     <input type="text" name="username" placeholder="Username" required>
13     <input type="password" name="password" placeholder="Password" required>
14
15     <button type="submit">Sign In</button>
16     <a href="{% url 'home' %}" class="back-link">Back</a>
17   </form>
18
19   <p class="small">
20     Don't have an account? <a href="{% url 'register' %}">Register</a>
21   </p>
22
```

About page

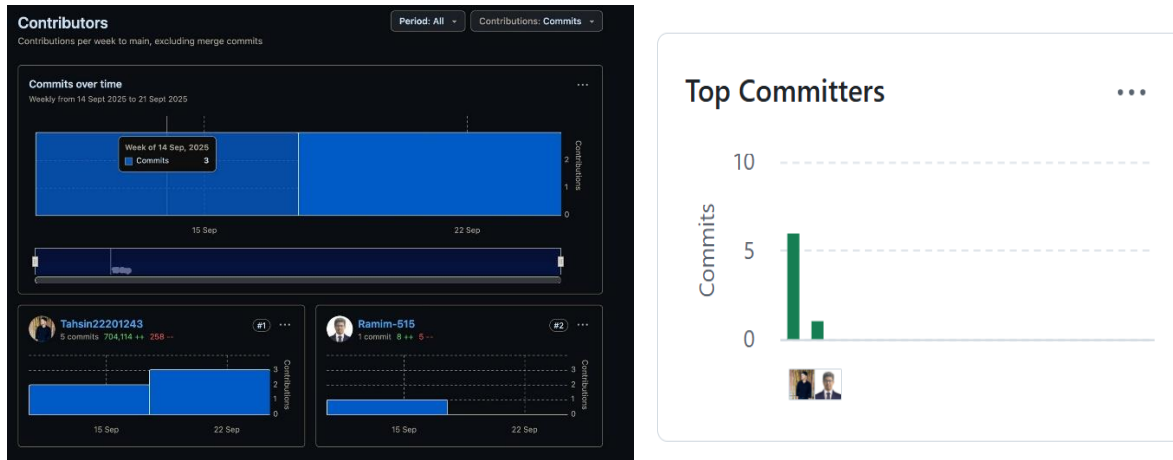


Contact page



8. Github Repository

The project is hosted on GitHub for version control and collaboration:



<https://github.com/Tahsin22201243/SohojWeb>

The repository includes both the backend and frontend code, along with project documentation and setup instructions.

9. Conclusion

This report outlines the initial phase of the Sohoj Biniyog project. The platform has successfully established the foundation for a Shariah-compliant investment system with a Django backend and HTML, CSS frontend. Core components such as backend setup, frontend integration, and basic navigation between pages have been implemented. Future enhancements will focus on enabling full campaign and investment management, secure authentication with role-based access, payment gateway integration, and a more interactive and user-friendly interface for investors and SMEs.